

Build Prompt: Campus Market

A student marketplace with login, Firebase storage, and buyer-seller DMs

Project Overview

Build a website where college students can buy and sell used study items: notes, calculators, textbooks, and previous year question papers. Visitors must log in before they can see or use the site. All data (accounts, listings, messages) is stored in Firebase. Buyers and sellers can privately message each other about a listing through a direct-message (DM) inbox built into the site.

Core Requirements

- Authentication gate: the whole site is login-only. Unauthenticated visitors are redirected to a login/sign-up page and cannot view listings or anything else.
- Sign up and log in with email + password (Firebase Authentication). Store a profile document per user (name, college, email) in Firestore on sign-up.
- Marketplace/browse page: a grid of listings, each showing title, category, description, price, and seller name.
- Categories: Notes, Calculator, Textbook, Previous Year Question Paper (PYQ), Other. Support filtering by category and a free-text search across title/description.
- Post an item: a logged-in user can create a listing with title, category, description, and price (0 = free). New listings appear for everyone in real time.
- Sellers can mark their own listings as sold / available again.
- Direct messages (DMs): from any listing, a buyer can click "Message seller" to open a private one-to-one chat with that seller. Each pair of users shares one running conversation (not one thread per item).
- Chat inbox: a page listing all of the current user's conversations (most recent first), each showing the other person's name, a preview of the last message, and which item it was about.
- Real-time messaging: messages sent in a conversation appear for both participants without a page refresh.
- Only the two participants in a conversation can read or send messages in it — enforce this with Firestore security rules, not just UI logic.
- Log out control available from every page once logged in.

Data Model (Firestore)

users/{uid} → name, college, email, createdAt

items/{itemId} → title, description, category, price, sellerId, sellerName, status (available/sold), createdAt

chats/{chatId} → participants[2 uids], participantNames, lastMessage, lastMessageTime, lastSenderId, lastReadBy, lastItemTitle (chatId = the two participants' uids, sorted and joined, so each pair has exactly one thread)

chats/{chatId}/messages/{messageId} → senderId, text, timestamp

Tech Stack

- Frontend: plain HTML/CSS/JavaScript (or React if preferred) — must work as a responsive, mobile-friendly site.
- Backend/data: Firebase Authentication (email/password) and Cloud Firestore for listings, chats, and messages. No custom backend server needed.
- Real-time updates: use Firestore's onSnapshot listeners for listings and messages so the UI updates live.
- Security: write and apply Firestore security rules so users can only edit their own profile/listings and can only read/write chats they are a participant in.

Pages

- Login / Sign up page — toggles between the two forms, redirects to the marketplace once authenticated.
- Marketplace page — search bar, category filter chips, listing grid, "+ Sell an item" button that opens a post form, and an item detail view with a "Message seller" action.
- Messages page — two-pane layout (conversation list + active chat) that collapses to a single pane with a back button on mobile.

Design Direction

Give it a distinctive "student marketplace" feel rather than a generic template look — think library index cards, notebook paper, stamped price tags. Suggested palette: warm paper background, deep ink-navy text, a highlighter-yellow accent, and a coral-red accent for tags/labels. Pair a characterful serif or display face for headings with a clean sans-serif for body text, and a monospace face for prices/category tags. Keep it readable and fast on mobile.

Nice-to-haves (optional, if time allows)

- Restrict sign-up to a specific college email domain.
- Password reset flow.
- Photo uploads for listings (Firebase Storage).
- Unread-message indicator in the nav.
- Push/browser notifications for new messages.